Estrutura de Dados II

Prof. Paulo Henrique Lopes Silva
UFERSA - DC

Árvores Rubro-Negras

Roteiro

- Introdução.
- Propriedades.
- Representação.
- Operações de busca e inserção.

Introdução

- Árvores rubro-negras são árvores binárias de busca.
  - Outros nomes: vermelho-preto ou *red black trees*.
- Foram criadas por **Bayer** com o nome de ***Árvores Binárias Simétricas*** em 1972.
- Em 1978, adquiriu seu atual nome por **Leo Guibas** e **Robert Sedgewick**.

Introdução

- Árvores Rubro-Negras possuem um **campo extra** para armazenar a **cor** de cada nó (vermelho ou preto).
- Uma estrutura comum de um nó em uma árvore rubro-negra:
  - **cor**: indica se o nó é vermelho ou preto.
  - **chave**: identificador do nó.
  - **valor**: valor associado do nó.
  - **dir** e **esq**: subárvores direita e esquerda.
  - **pai**: armazena o ponteiro ou referência para o pai do nó.
  - Se o filho ou pai não existir: `null`.

Introdução

- Árvores Rubro-Negras.
  - São estruturas mais complexas que árvores binárias de busca simples, pois exigem regras adicionais de balanceamento.
    - Coloração dos nós (vermelho/preto), regras estruturais que limitam a altura.
    - **Balanceamento**: Rotações e recoloração de nós.
  - Garantem bom desempenho mesmo no pior caso, evitando degradação para listas encadeadas.
    - **Eficiente**: busca, inserção e remoção em ***O(log n)***, sendo ***n*** o número de elementos da árvore.
  - **Resultado:** é uma ABB que realiza as operações de forma eficiente e previsível para permanecer balanceada.

Propriedades

- Árvores Rubro-Negras possuem as seguintes propriedades:

  1. Todo nó deve possuir coloração **vermelha** ou **preta**.
  2. A raiz é **preta**.
  3. Toda folha (`null`) é **preta**.
  4. Se um nó é **vermelho**, então seus filhos são **pretos**.
  5. Para cada nó, **qualquer caminho** deste nó para as folhas descendentes contém o **mesmo número de nós pretos**.

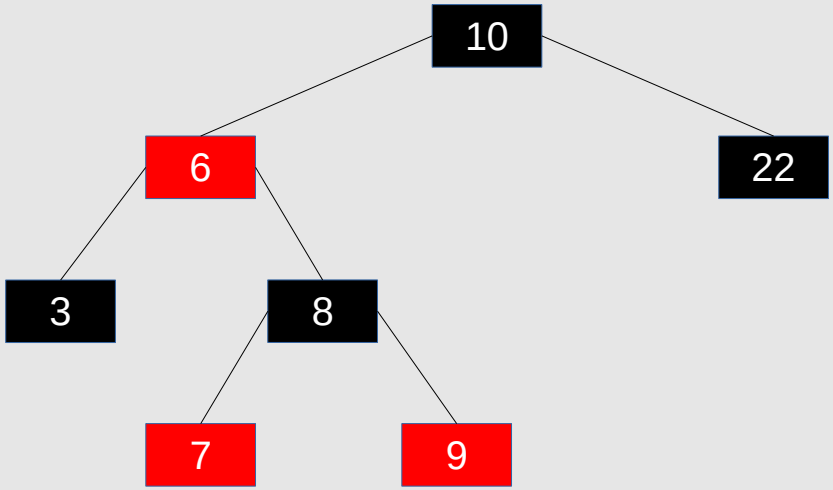
Propriedades

- Nós que satisfazem as propriedades citadas são equilibrados (balanceados).
  - Caso contrário: desequilibrado (desbalanceado).
- Consequências das propriedades:
  - Não existem dois nós vermelhos consecutivos.
    - Em qualquer caminho da raiz até uma folha (nó nulo), um nó vermelho não pode ter pai vermelho.
    - Consequência:
      - Evita o crescimento descontrolado da árvore.
      - Garante uma distribuição mais uniforme dos nós.

Propriedades

- Consequências das propriedades:
  - Altura aproximadamente equilibrada entre caminhos.
    - O maior caminho da raiz até uma folha é, no máximo, duas vezes maior que o menor caminho.
    - Consequência:
      - A árvore não se torna extremamente desbalanceada.
      - Mantém eficiência nas operações.

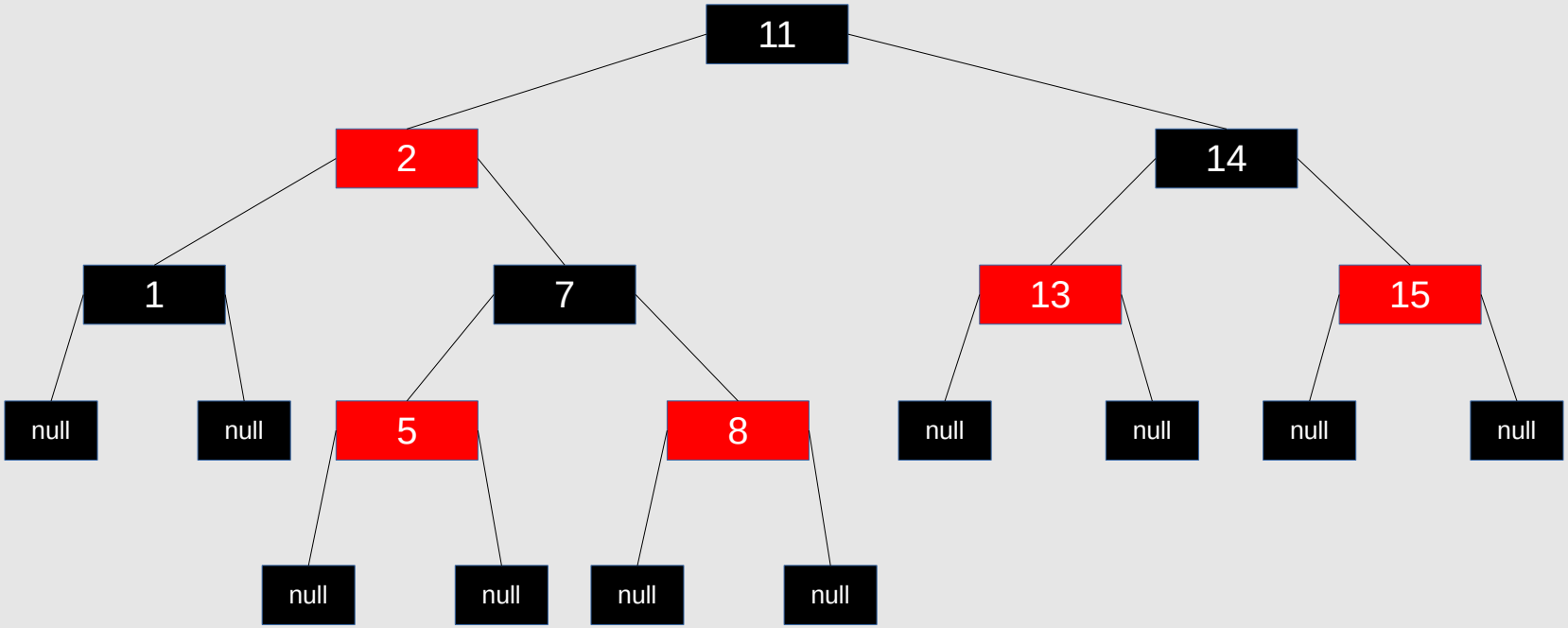
- Caminhos da raiz até as folhas.
  - Menor: 22 - null
  - Maior: 6 - 8 - (7 ou 9) - null



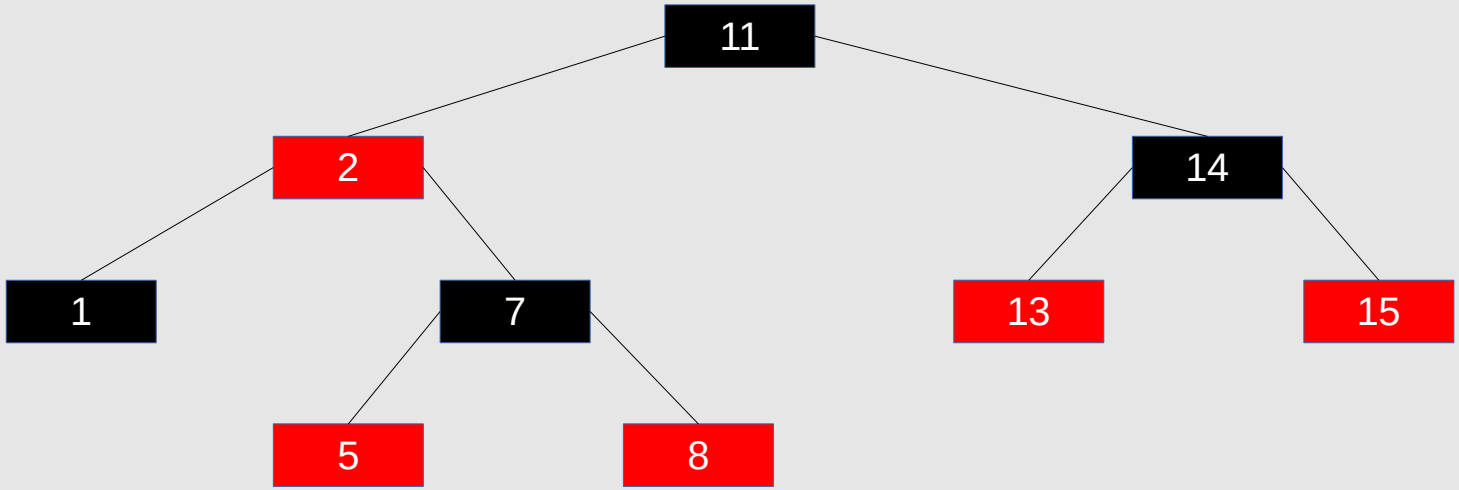
Propriedades

- Consequências das propriedades:
  - Altura limitada por O(log n).
    - A altura da árvore cresce de forma logarítmica em relação ao número de elementos.
    - Mesmo após várias inserções e remoções, essa propriedade é mantida.
    - Consequência: Operações como busca, inserção e remoção permanecem eficientes.
  - Balanceamento aproximado (não perfeito).
    - A árvore não é perfeitamente balanceada (como AVL), mas mantém um equilíbrio suficiente.
    - Consequência:
      - Menor número de rotações comparado a AVL.
      - Melhor desempenho em cenários com muitas inserções e remoções.

Representação



Representação

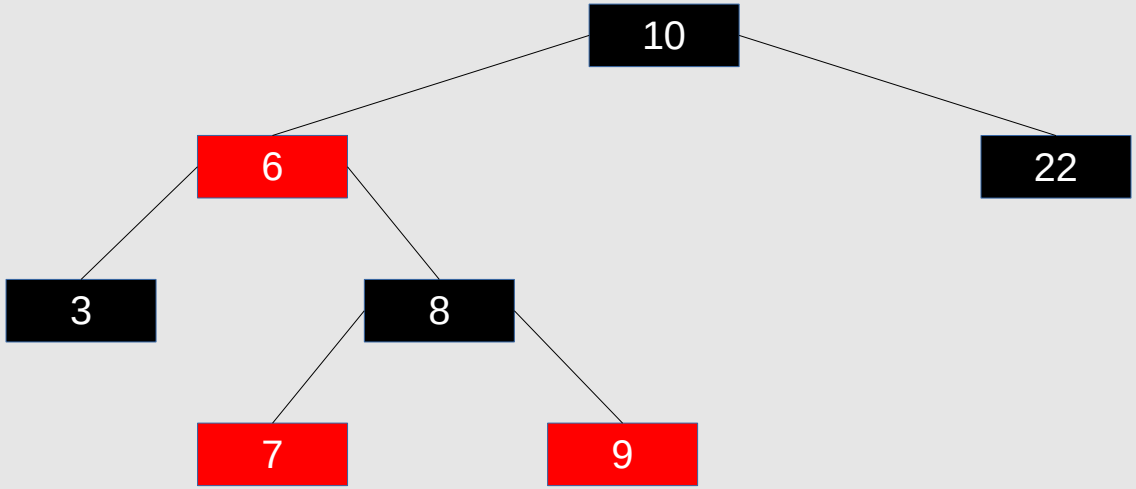


Propriedades

- **Altura de preto** de um nó **n** é o número de nós pretos (sem incluir o **n**) que se encontram nos caminhos de **n** até as folhas.
  - Pela regra 5, $h_p(n)$ é bem definida para todos os nós.
  - A altura de preto da árvore rubro-negra é $h_p(raiz)$.
  - Pode-se considerar os nós `null` na contagem da altura de preto ou não.

Propriedades

- Altura de preto da árvore?

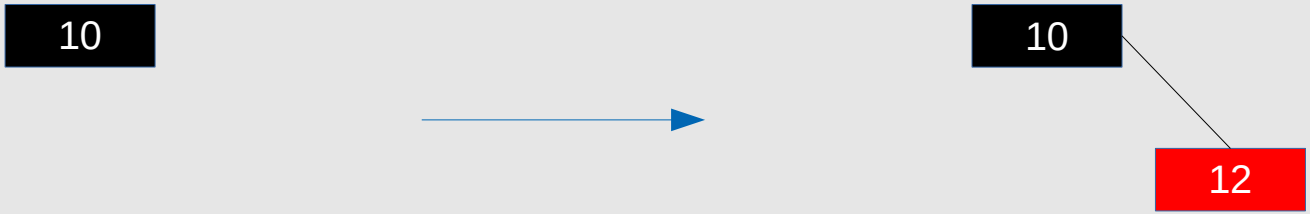


Operações

- A busca em uma ABB funciona **sem modificações**.
- As operações de inserir e remover podem ser implementadas de forma parecida com as das ABBs.
  - A diferença reside em sempre verificar se a operação realizada fere as propriedades da árvore rubro-negra.
  - São mais critérios a serem analisados em relação a árvore AVL.

Inserção em árvores rubro-negras

- Todo nó a ser inserido é vermelho (inserções não devem alterar a **altura de preto** da árvore).
- Exemplo:

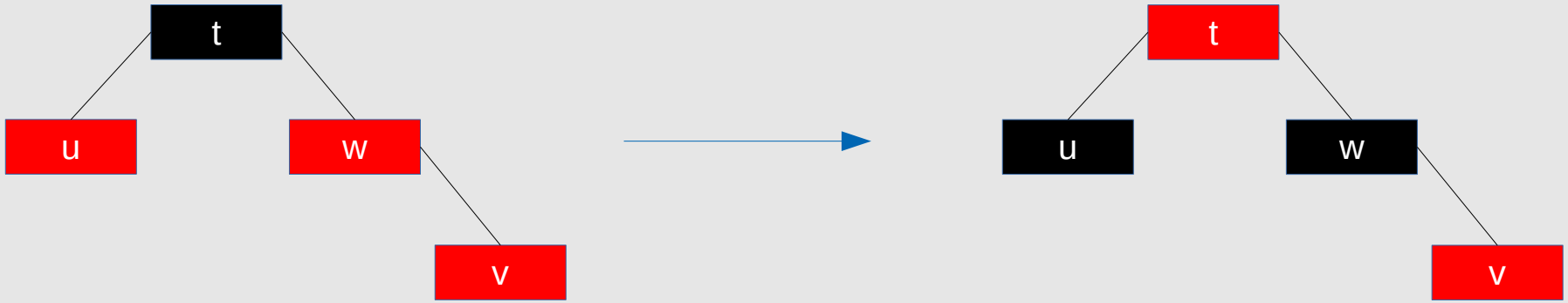


Inserção em árvores rubro-negras

- A inclusão de novos nós pode resultar em alguns ajustes.
  - Destacam-se dois casos.

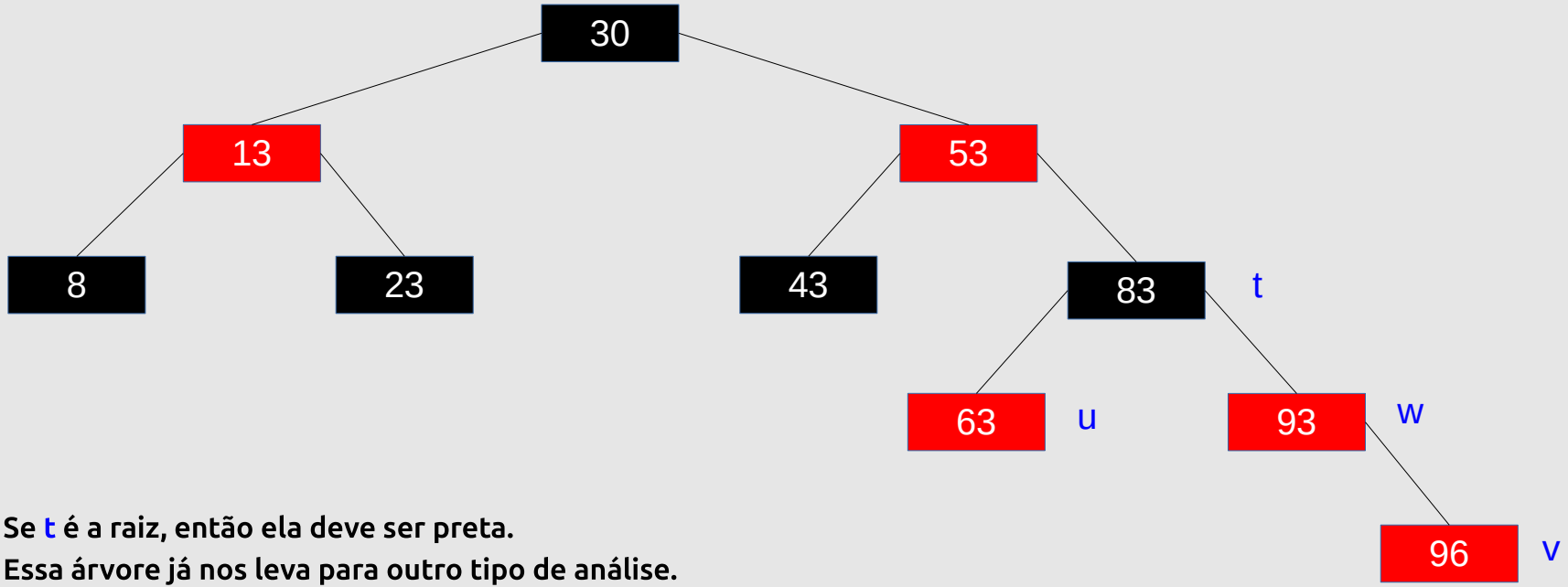
Inserção em árvores rubro-negras

- **Caso 1**: seja w vermelho e t, o pai de w (avô de v) preto. Se u, o irmão de w (tio de v) é vermelho, é possível manter a regra 4 fazendo a recoloração de t, u e w.



Inserção em árvores rubro-negras

- Se o pai de t for vermelho, repete-se o processo com v = t.



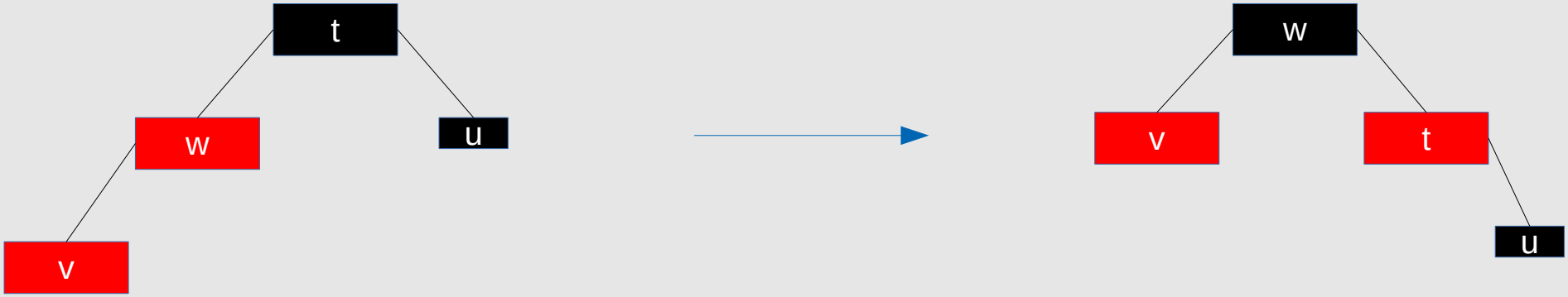
Obs1: Se t é a raiz, então ela deve ser preta.
Obs2: Essa árvore já nos leva para outro tipo de análise.

Inserção em árvores rubro-negras

- Sobre o nó v:
  - Se seu pai w é vermelho, seu avô t preto e seu tio u preto. Neste caso, para manter a regra 4 é preciso fazer rotações envolvendo w, t, u, v.
  - Existem 4 subcasos, com 4 rotações possíveis.

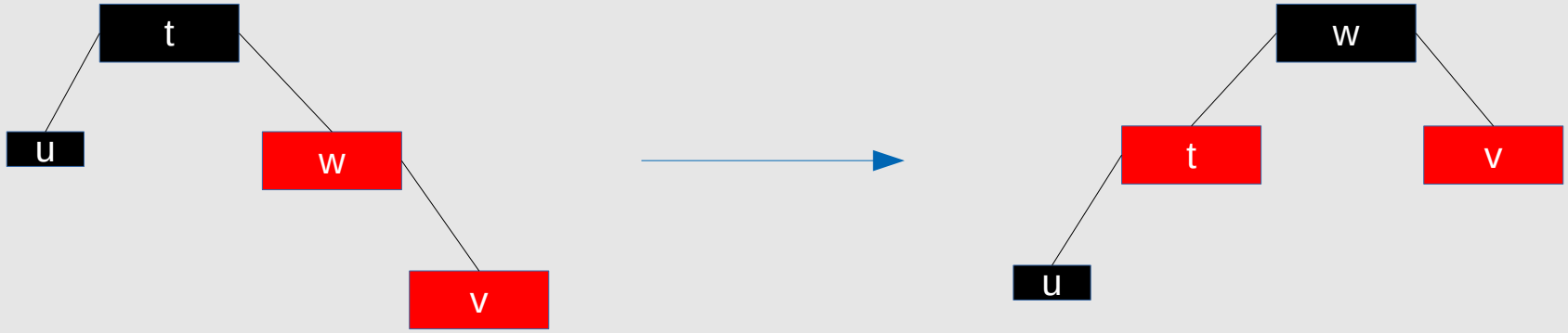
Inserção em árvores rubro-negras

- **Caso 2a**: pai w do novo nó v é filho esquerdo e o novo nó é filho esquerdo.
  - Pai w muda para preto.
  - Avô t muda para vermelho.
  - Rotação direita simples no avô t.



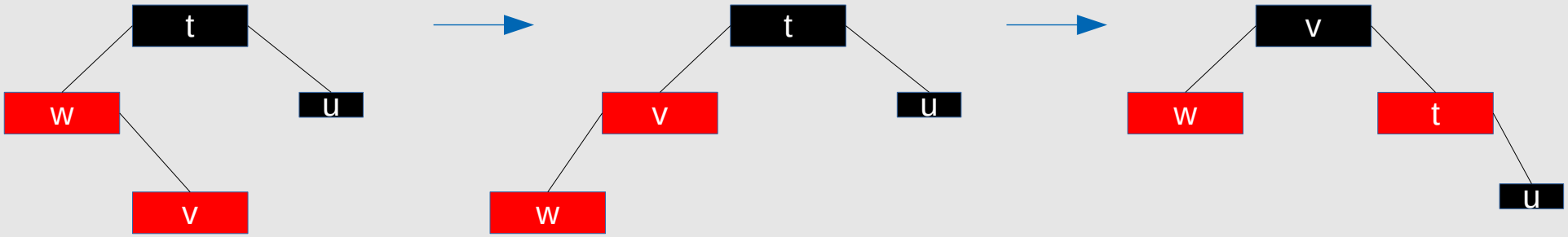
Inserção em árvores rubro-negras

- **Caso 2b**: pai w do novo nó v é filho direito e o novo nó é filho direito.
  - Pai w muda para preto.
  - Avô t muda para vermelho.
  - Rotação esquerda simples no avô t.



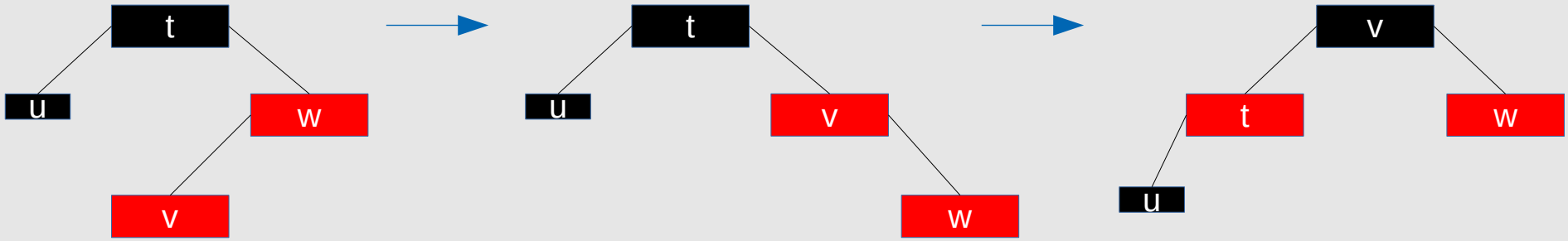
Inserção em árvores rubro-negras

- **Caso 2c**: Se v é filho direito de w e w é filho esquerdo de t (RDD):
  - RES no pai w.
  - Novo nó v = filho esquerdo do novo nó (w).
  - Pai muda para preto.
  - Avó muda para vermelho.
  - RDS no avô.



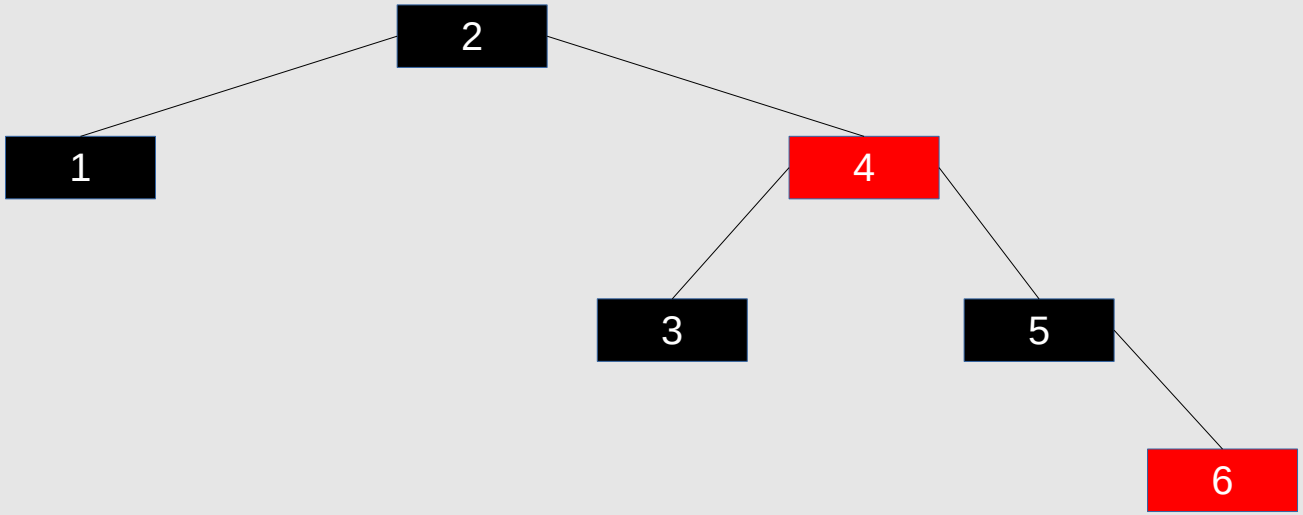
Inserção em árvores rubro-negras

- **Caso 2d**: Se v é filho esquerdo de w e w é filho direito de t (RDE):
  - RDS no pai w.
  - Novo nó v = filho direito do novo nó (w).
  - Pai muda para preto.
  - Avó muda para vermelho.
  - RES no avô.



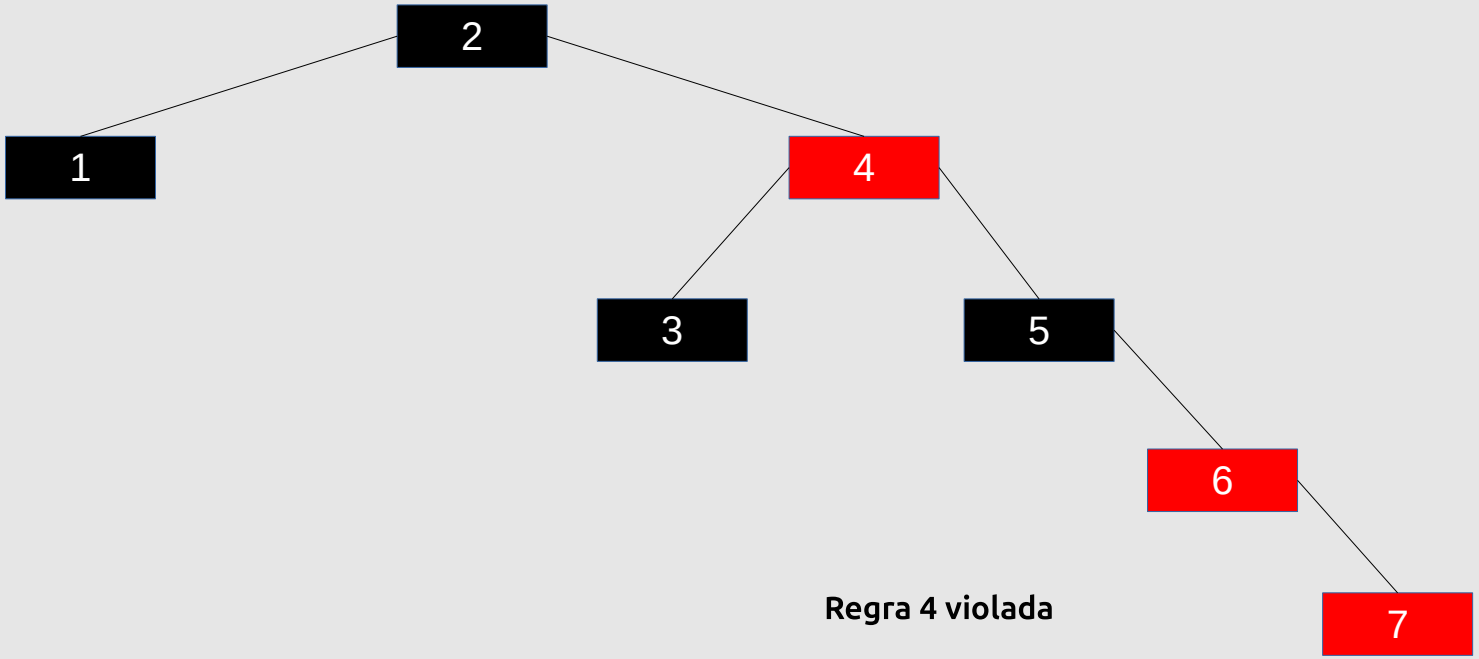
Inserção em árvores rubro-negras

- Exemplo:



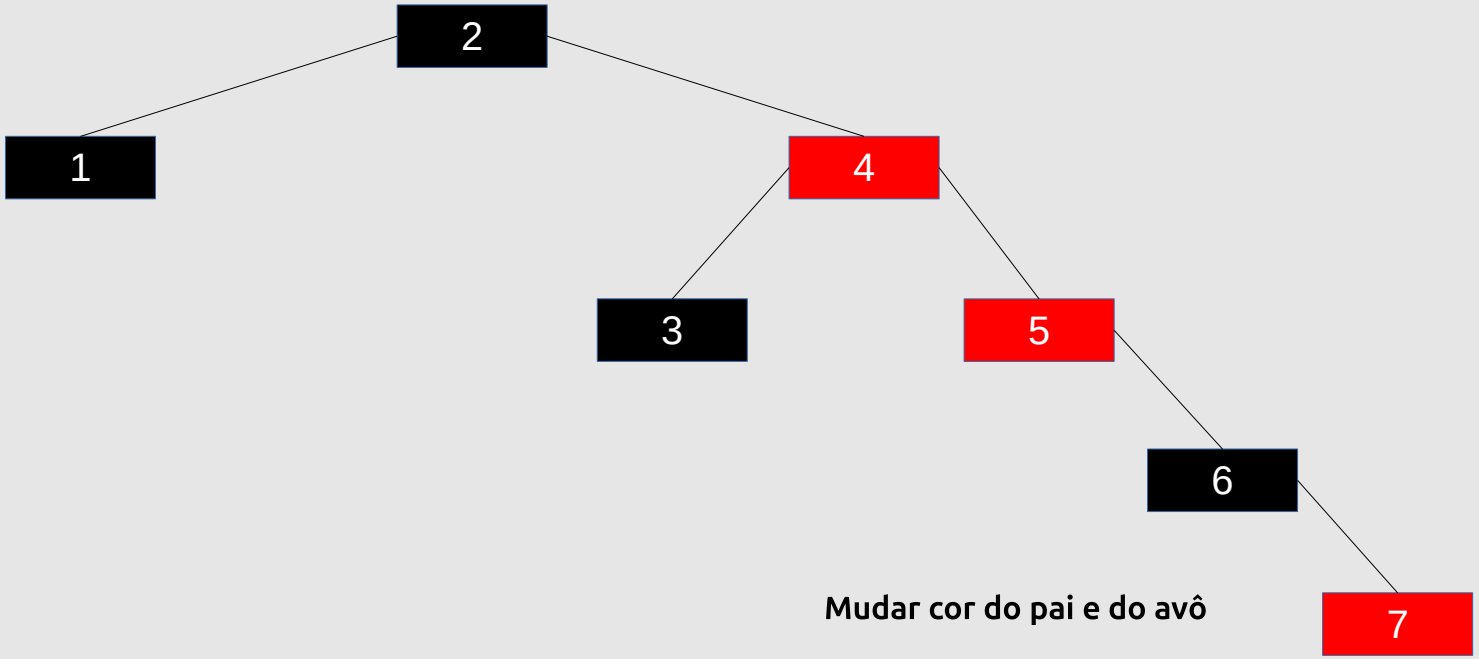
Inserção em árvores rubro-negras

- Exemplo: inserindo o 7.



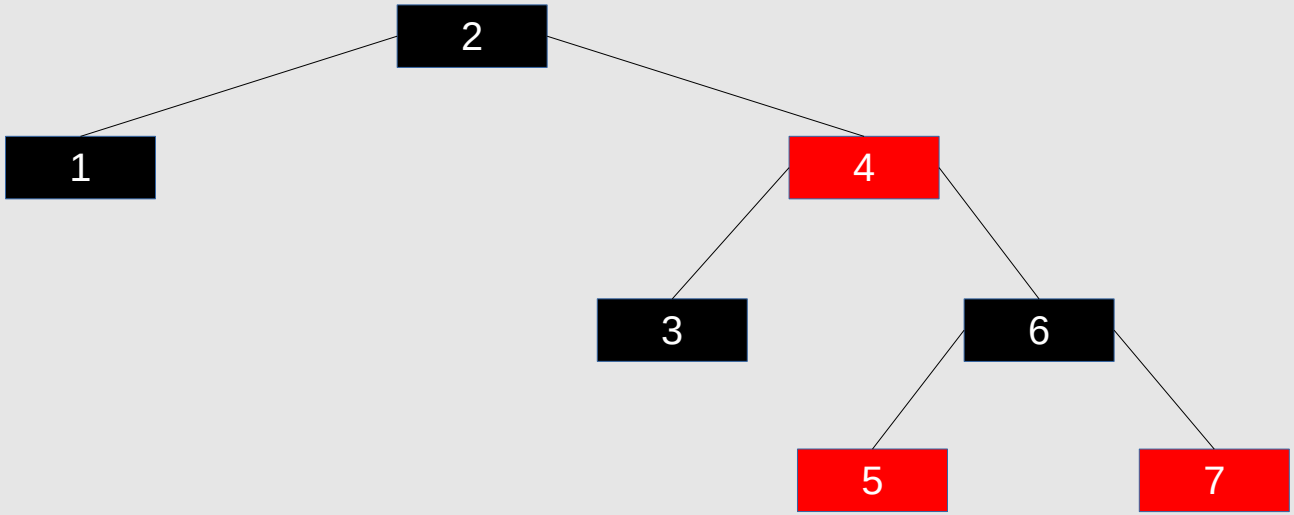
Inserção em árvores rubro-negras

- Exemplo: rotação esquerda simples com 5,6 e 7.



Inserção em árvores rubro-negras

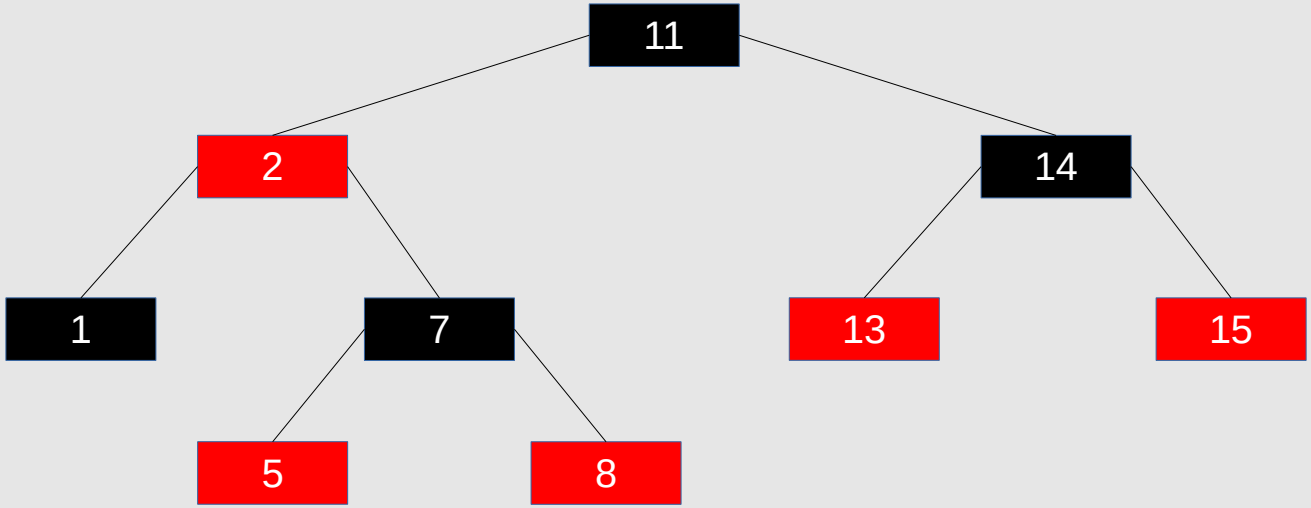
- Exemplo: rotação esquerda simples com 5,6 e 7.



Resultado da RES.

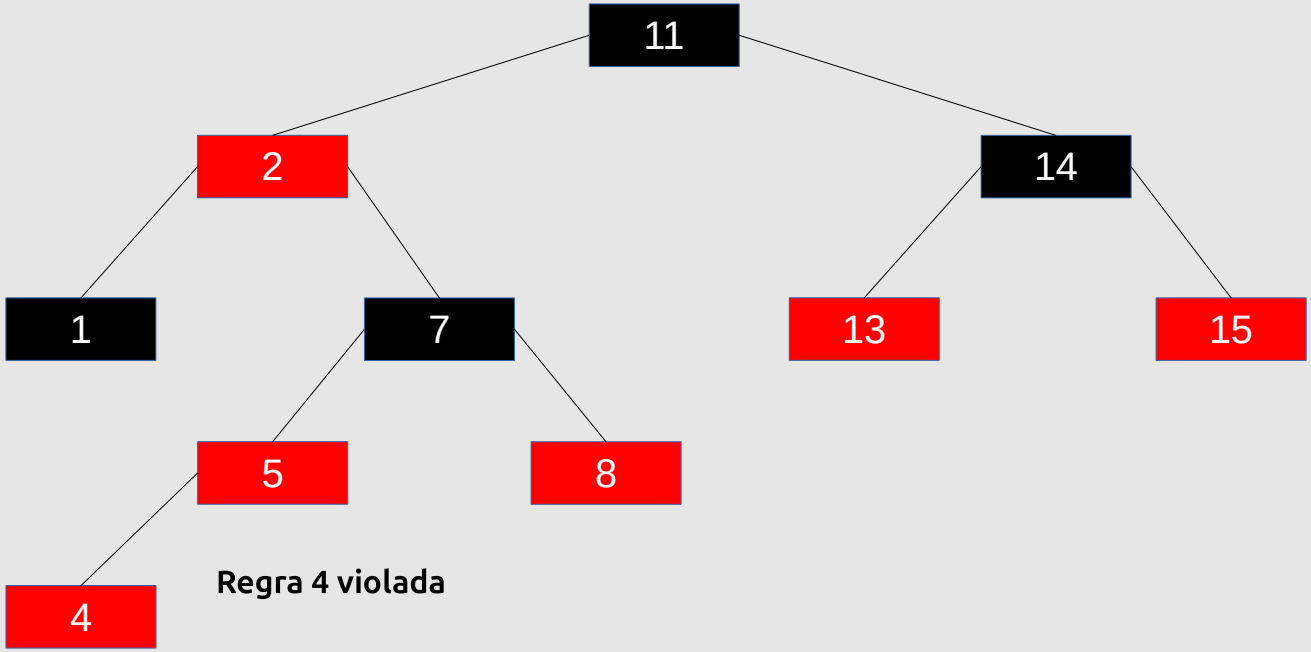
Inserção em árvores rubro-negras

- Exemplo 2:



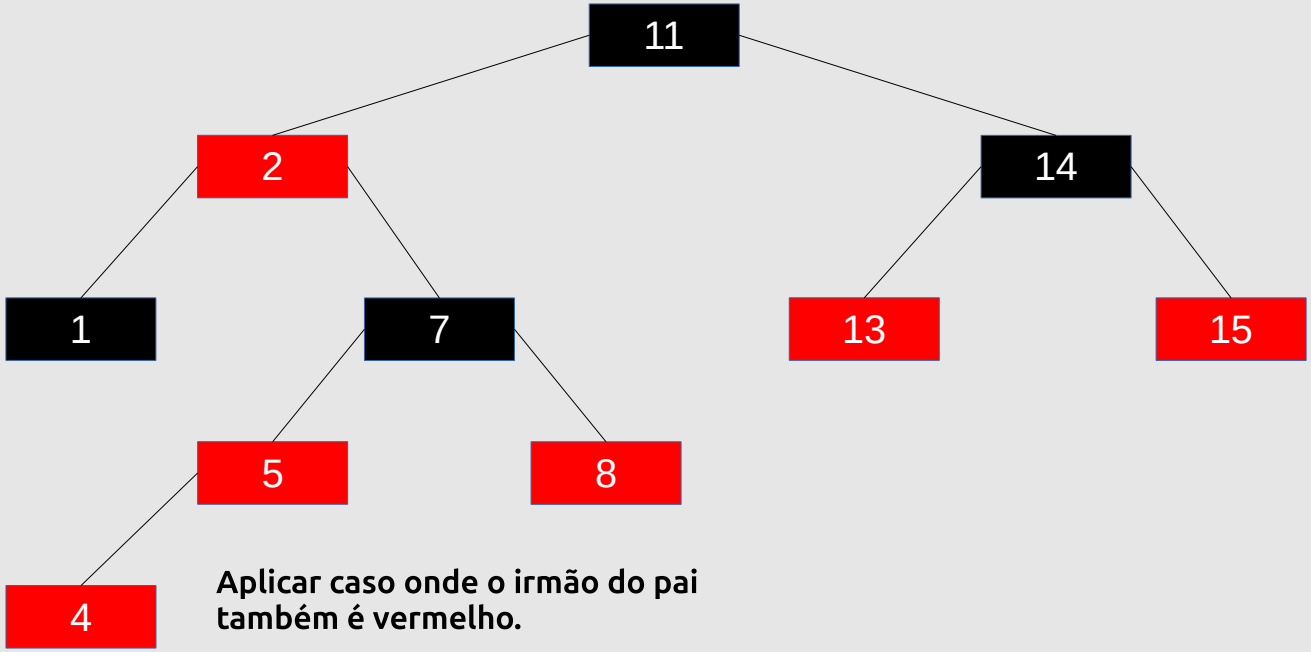
Inserção em árvores rubro-negras

- Exemplo 2: inserindo o 4.



Inserção em árvores rubro-negras

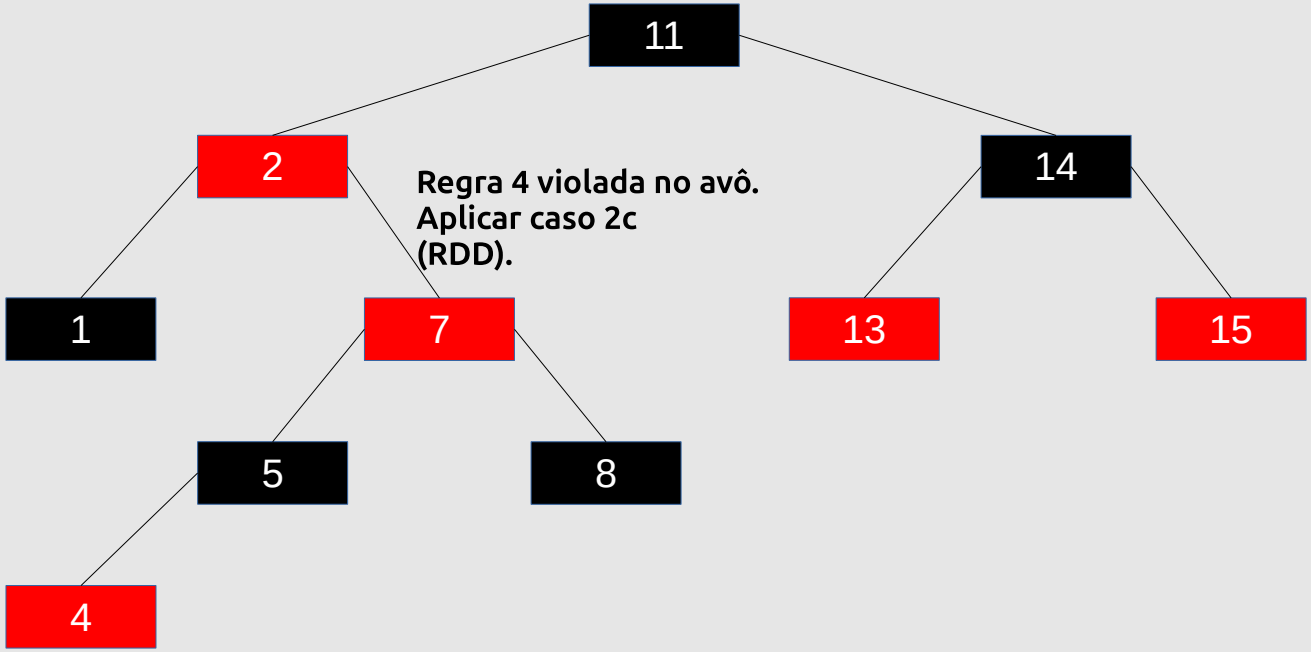
- Exemplo 2: inserindo o 4.



Aplicar caso onde o irmão do pai também é vermelho.

Inserção em árvores rubro-negras

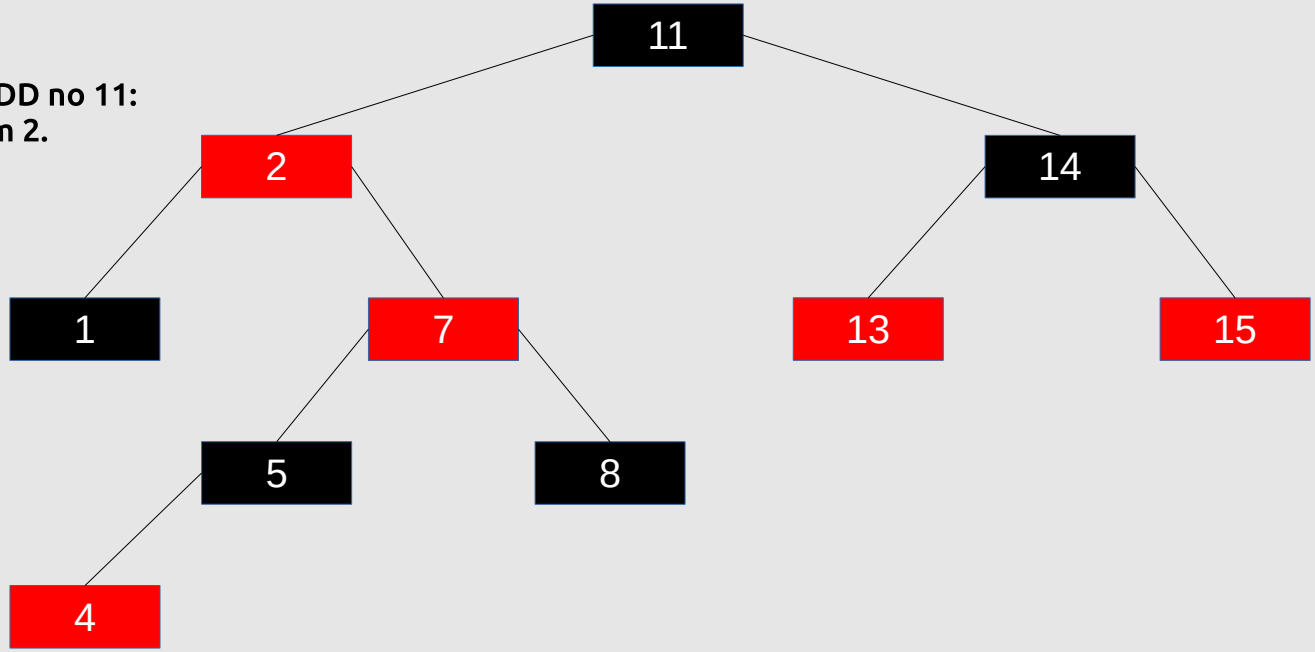
- Exemplo 2: aplicando o caso 1 (irmão do pai também é vermelho).



Inserção em árvores rubro-negras

- Exemplo 2: aplicando o caso 2c (RDD).

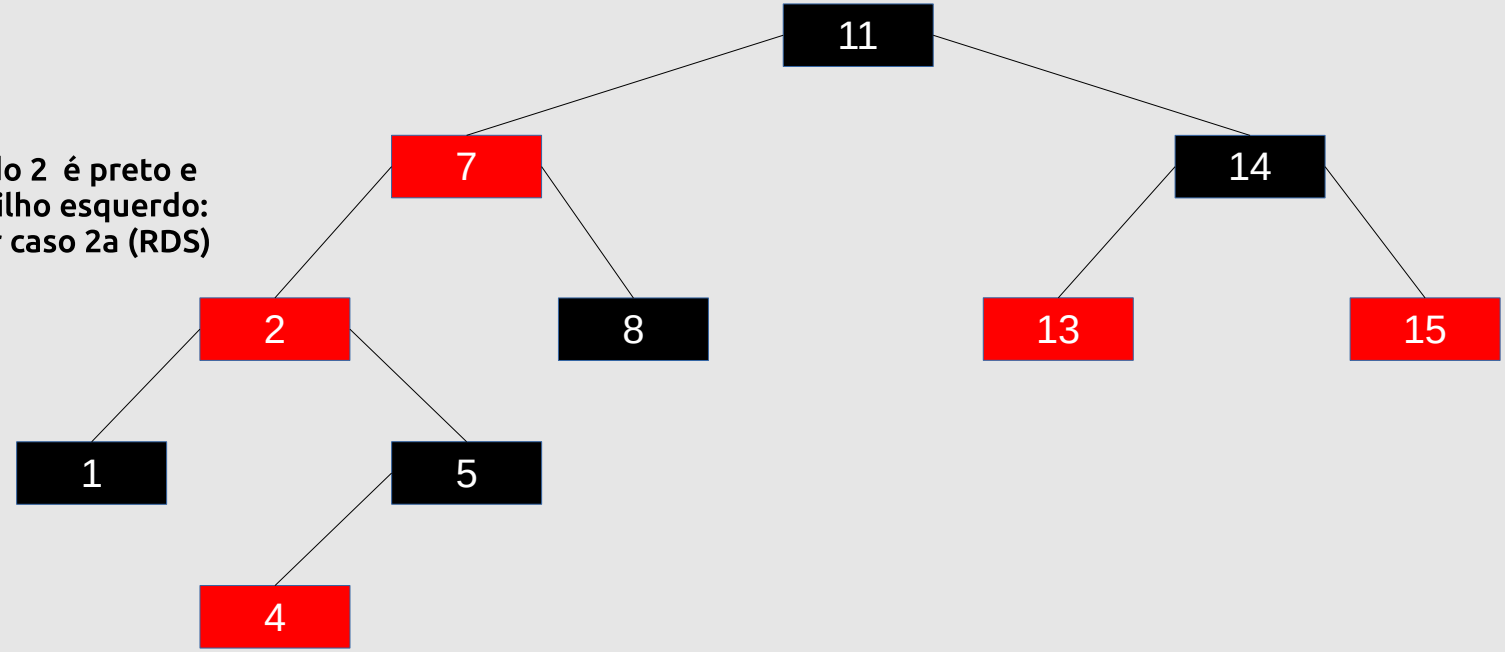
1º passo da RDD no 11: aplicar RES em 2.



Inserção em árvores rubro-negras

- Exemplo 2: RES em 2 aplicado.
  - Agora, aplicar RDS em 11.

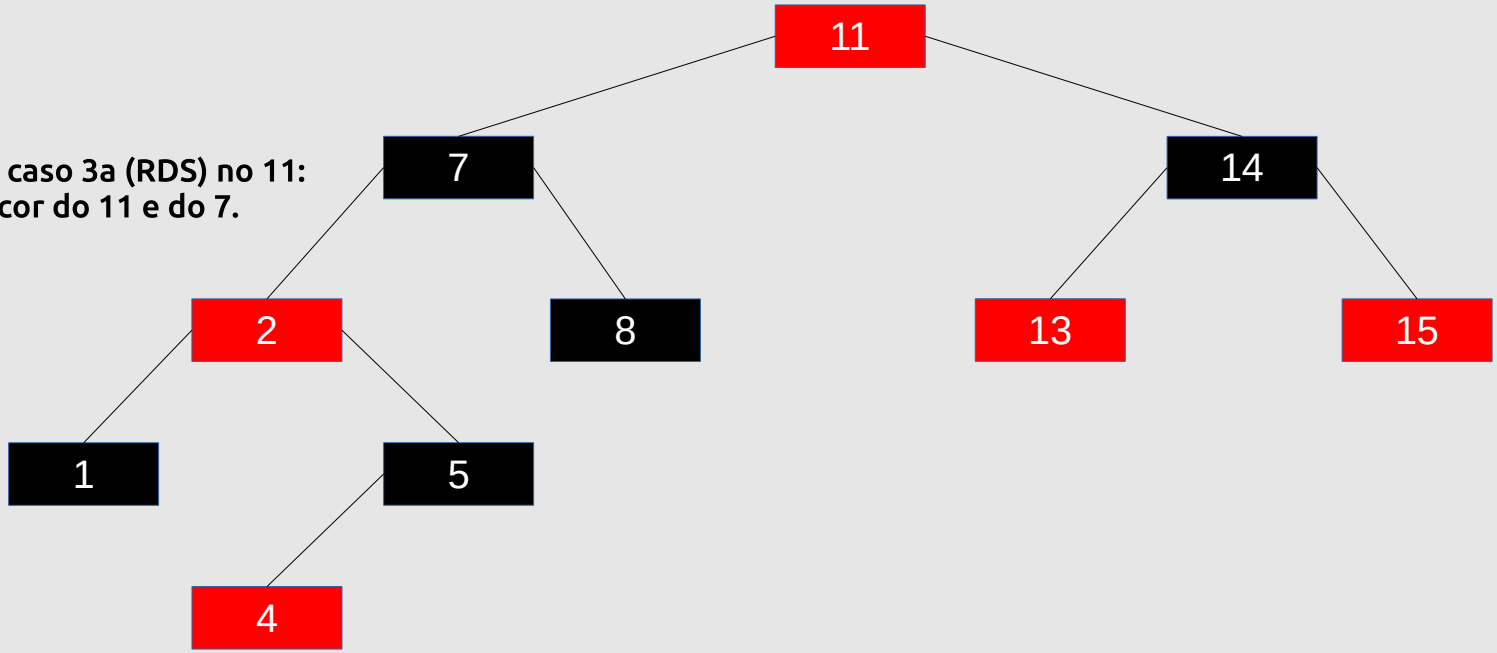
O tio do 2 é preto e
Ele é filho esquerdo:
aplicar caso 2a (RDS)



Inserção em árvores rubro-negras

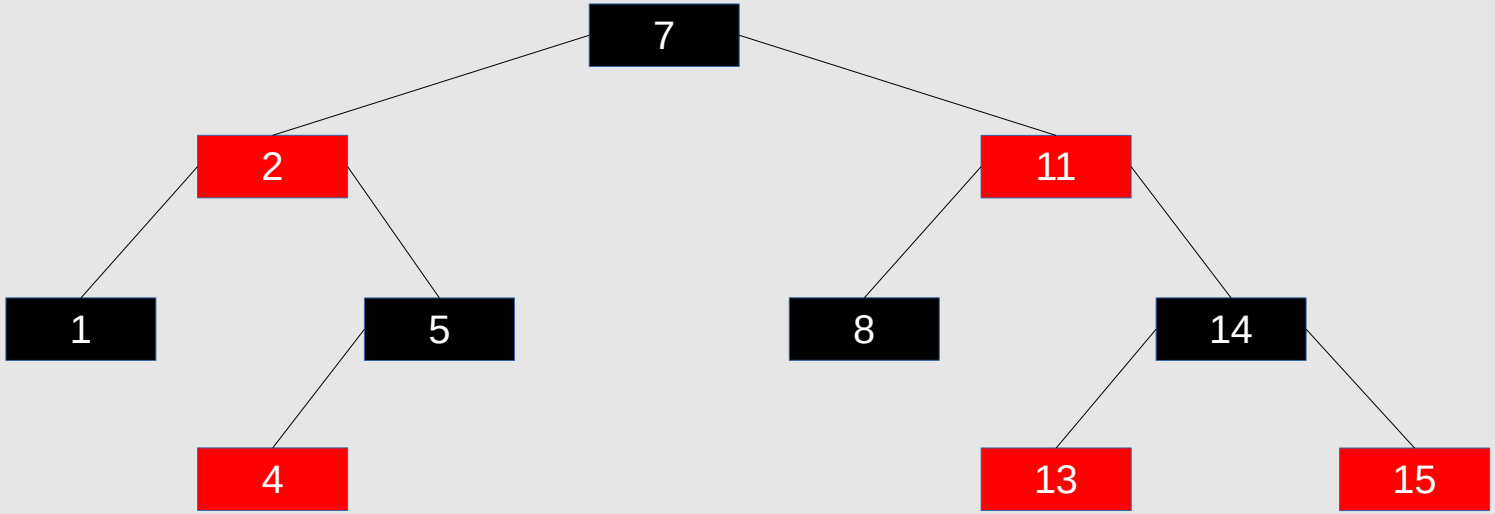
- Exemplo 2: RES em 2 aplicado.
  - Agora, aplicar RDS em 11.

Aplicar caso 3a (RDS) no 11: mudar cor do 11 e do 7.



Inserção em árvores rubro-negras

- Exemplo 2: resultado da aplicação do caso 2a (RDS) no 11.



- **Com a RDS realizada, o filho direito do 7 passa a ser filho esquerdo do 11.**
- **A regra 4 é restaurada e nenhuma outra é violada.**

Inserção em árvores rubro-negras

- Exercício: inserir as chaves 61, 58, 51, 32, 39, 28.

Referências

- Principais referências.

  - CORMEN, Thomas H et al. **Algoritmos**: teoria e prática. 2. ed. Rio de Janeiro: Campus, 2002.
  - SZWARCFITER, Jayme Luiz; MARKENZON, Lilian. **Estruturas de dados e seus algoritmos**. 3.ed. Rio de Janeiro: LTC, 2010.
  - SEDGEWICK, Robert; WAYNE, Kevin. **Algorithms**. 4th Edition. Addison-Wesley, 2011.